

Sovereign Process Mining: Bridging the Gap Between Event Data and Cryptographic Execution

Wil M.P. van der Aalst

*Process and Data Science (PADS), RWTH Aachen University
Aachen, Germany*

wvdaalst@pads.rwth-aachen.de

June 5, 2026

Abstract

Process mining provides the critical link between data science and business process management, enabling organizations to discover, monitor, and improve real processes by extracting knowledge from event logs. However, as Process-Aware Information Systems (PAIS) decentralize into edge, autonomic, and trustless environments, the traditional paradigms of centralized process mining become insufficient. In this paper, we present **wasm4pm**, a sovereign process mining ecosystem compiled to WebAssembly. We formally define the integration of classical process mining concepts—such as token-based replay, A* alignment for conformance checking, and inductive discovery over Partially Ordered Workflow Languages (POWL)—with strict typestates and continuous cryptographic provenance. By embedding process execution inside a verifiable, bounded, and deterministic WebAssembly container, we show how process mining can transition from an a-posteriori auditing tool into a foundational, mathematically rigorous execution authority for decentralized systems.

1 Introduction

Modern organizations are driven by Process-Aware Information Systems (PAIS), ranging from traditional ERP systems (e.g., SAP, Oracle) to custom distributed microservices. These systems generate massive amounts of event data. Process mining acts as the bridge between traditional model-driven business process management (BPM) and data-driven machine learning, providing techniques to discover the actual “as-is” processes, check conformance against prescribed models, and enhance existing processes [?].

The classical process mining framework relies on three fundamental capabilities:

1. **Play-In (Discovery):** Extracting a process model (e.g., a Petri net, Directly-Follows Graph, or Process Tree) from an event log.
2. **Play-Out (Simulation):** Generating behavior (i.e., traces) from a process model to explore potential scenarios.
3. **Replay (Conformance Checking):** Replaying real event data on top of a process model to detect deviations, bottlenecks, and compliance violations.

Despite the maturity of these algorithms, traditional process mining tools assume a centralized, trusted execution environment where event logs are passively analyzed in a data warehouse. This paradigm is fundamentally misaligned with the emerging landscape of decentralized, trustless, and highly autonomic mesh networks.

In this paper, we introduce **wasm4pm**, a process mining framework that embeds rigorous process execution and analysis within a WebAssembly (WASM) engine. We argue that process

models should not merely be descriptive or predictive artifacts, but rather *executable authorities*. We demonstrate how **wasm4pm** enforces the formal semantics of Petri nets and POWL (Partially Ordered Workflow Language) through strict tpestates, ensuring that every algorithmic claim is bound to a cryptographic receipt.

2 Formal Foundations of Event Logs and Process Models

To ground our discussion, we first formalize the core concepts of process mining, adapting them for the verifiable boundaries of the **wasm4pm** ecosystem.

Definition 1 (Event, Trace, and Event Log). *Let $\mathcal{A}$ be a universe of activity names, $\mathcal{C}$ be a universe of case identifiers, $\mathcal{T}$ be a domain of timestamps, and $\mathcal{D}$ be a universe of additional attributes.*

- An **event** $e = (c, a, t, d)$ is a tuple where $c \in \mathcal{C}$, $a \in \mathcal{A}$, $t \in \mathcal{T}$, and $d \subseteq \mathcal{D}$. The projection functions $\pi_c(e)$, $\pi_a(e)$, $\pi_t(e)$ yield the case, activity, and timestamp respectively.
- A **trace** $\sigma = \langle e_1, e_2, \dots, e_n \rangle$ is a finite sequence of events such that $\forall i, j \in \{1, \dots, n\} : \pi_c(e_i) = \pi_c(e_j) \text{ and } i < j \implies \pi_t(e_i) \leq \pi_t(e_j)$. We denote the sequence of activities as $\bar{\sigma} = \langle \pi_a(e_1), \dots, \pi_a(e_n) \rangle$.
- An **event log** L is a multiset of traces.

In **wasm4pm**, an event log is not merely a raw XML or JSON file. It must cross the **wasm4pm-compat** *Court of Admissibility*, transitioning from L_{raw} to $L_{admitted}$. This ensures that all events conform strictly to the XES_{1849} or $OCEL_{2.0}$ witness constraints before any algorithmic execution occurs.

2.1 Process Modeling via Petri Nets

While there are many process modeling notations (BPMN, EPCs, UML), we utilize Petri nets due to their unambiguous formal semantics.

Definition 2 (Marked Petri Net). *A Petri net is a tuple $N = (P, T, F)$ where P is a finite set of places, T is a finite set of transitions ($P \cap T = \emptyset$), and $F \subseteq (P \times T) \cup (T \times P)$ is a set of directed arcs. A **marking** M is a multiset over P , representing the state of the net. A marked Petri net is a pair (N, M_{init}) where M_{init} is the initial marking.*

Within **wasm4pm**, Petri nets are represented via the **FlatIncidenceMatrix**. For performance-critical WebAssembly execution, the adjacency is flattened into contiguous memory arrays, allowing $O(1)$ state updates during execution. Transitions $t \in T$ are partitioned into visible transitions T_V (mapped to A) and silent/invisible transitions T_τ .

3 Conformance Checking via Replay and Alignments

Conformance checking compares an event log L with a process model (N, M_{init}) . Early approaches relied on naive token-based replay, which struggles with "spaghetti models" and severe deviations, often flooding the net with missing and remaining tokens.

Modern process mining solves this using **Alignments** [?]. An alignment maps a trace σ to a valid execution path σ_M in the model, identifying synchronous moves, log moves (deviations in reality), and model moves (deviations in the model).

Definition 3 (Alignment). *Let $\sigma \in L$ and $N = (P, T, F)$. A legal move is a pair (x, y) where:*

- (a, a) is a **synchronous move** if $a \in \bar{\sigma}$ and $t \in T_V$ with $l(t) = a$.

- $(a, \gg)$ is a **log move** (activity a occurred but is not allowed by the model).
- $(\gg, t)$ is a **model move** (transition t fires in the model, but was not observed).

An alignment γ is a sequence of legal moves such that the projection on the first element yields $\bar{\sigma}$ and the projection on the second element yields a valid firing sequence σ_M from M_{init} to a final marking M_{final} .

The **wasm4pm** engine utilizes an A^* search algorithm over the state space of the synchronous product of the trace and the model. To guarantee performance within the bounded WebAssembly environment, the state equation heuristic solves a relaxed linear program (LP) over the **FlatIncidenceMatrix**. The cost function $\kappa(\gamma)$ assigns strictly positive costs to asynchronous moves: $\kappa(a, \gg) > 0$ and $\kappa(\gg, t) > 0$.

4 Sovereign Discovery and Partially Ordered Workflows (POWL)

Process discovery algorithms (e.g., the Alpha algorithm, Heuristics Miner, Inductive Miner) must balance four competing quality criteria: **Fitness** (ability to replay the log), **Precision** (avoiding underfitting), **Generalization** (avoiding overfitting), and **Simplicity** (Occam’s razor).

The Inductive Miner is a fundamental breakthrough because it guarantees the discovery of sound process models by recursively applying graph cuts on a Directly-Follows Graph (DFG). In **wasm4pm**, we extend this capability by generating Partially Ordered Workflow Languages (POWL) via the **PowlArena**.

Theorem 1 (POWL to Petri Net Projection). *Let P be a POWL specification. If P contains an irreducible **ChoiceGraph** construct $\mathcal{G}$, **wasm4pm** projects this onto a behaviorally isomorphic Petri net PN by synthesizing silent single-entry single-exit (SESE) τ -transitions. This ensures that the generated language $L(P) \subseteq L(PN)$ is preserved monotonically, maintaining strict block-structured soundness where possible, and bounded non-block-structured routing where necessary.*

5 Conclusion

Process mining is no longer confined to retrospective analysis on centralized servers. The future of Process-Aware Information Systems requires bringing process intelligence directly to the edge. The **wasm4pm** framework achieves this by wrapping the formal mathematics of Petri nets, alignments, and process discovery within a verifiable, WebAssembly-bound execution envelope. By strictly separating the *Court of Admissibility* (**wasm4pm-compatible**) from the *Execution Authority*, and binding all outputs to cryptographic BLAKE3 receipts, we elevate process models from mere diagnostic tools to sovereign, trustless arbiters of organizational behavior.

References

- [1] W.M.P. van der Aalst. *Process Mining: Data Science in Action*. Springer, 2nd Edition, 2016.
- [2] A. Adriansyah, B.F. van Dongen, and W.M.P. van der Aalst. ”Conformance Checking using Cost-Based Fitness Analysis”. *IEEE EDOC*, 2011.